MemDB2 Codebase Review: Master Issue List (updated 3 Nov 2026, rev. 8)

Scope: all 30 units in Codebase_for_claude.zip (about 24k lines excluding the test form): MemDB2 (core, API, buffering, indexing, journal, streaming), IndexedStore, BinaryTree, CoWTree, Reffed, Trackables, RWWLock/TinyLock/StripedLock/SRWLock, DLList/DLThreadQueue, Parallelizer, BufferedFileStream, NullStream, CRC32, PComp, PatchFreeMem. Line numbers refer to the source files. Release builds have **assertions disabled** (DCC_AssertionsAtRuntime=false), so every Assert-only check below is absent in production. 'Latent' means the defect exists but no current MemDB2 code path reaches it.

Category	High	Medium	Low / Low-Med	Info	Total open	Resolved
1. Bugs (incl. Round 10 items)	0	3	9	2	14	13
Won't fix / withdrawn	-	-	-	-	11 / 1	-
2. Optimisations	2	4	8	0	14	0
3. Not worth fixing (already handled)	-	-	-	-	17	-

What changed since the first review: stream I/O moved to ReadBuffer/WriteBuffer throughout; failed flushes now raise; journal and checkpoint files are written as *_partial, flushed, then renamed; NullStream.Write returns Count. IDs are kept from the first report; N-numbered items are new (N1/N2 come from the temp-file scheme, N3-N5 from the cached-stream follow-up). Since v2: the journal lifecycle diff resolved B2, B5 and B19, the RWWLock diff resolved B8 and B12, and the 11 open items from your Round 10 report, each re-checked against the current code and merged into the bug list with an R10- prefix (10 still open, Q17 half done). Duplicates were combined: R10-Q25 into B14, R10-P2 into optimisation O1.

This revision: B18, R10-Q16 and R10-N2 resolved; B14/R10-Q25, B20 and B22 partly fixed; B21 withdrawn (my error); 11 items marked won't fix at your request (section 1c). Rev. 7: B14 notes updated (size reset not needed). Rev. 8: B20 resolved (GetInitialFileList leaks; FreeRList nil-safe); its StrToInt64 point split out as B20b (Info). **Fix next:** N3 (stale read cache can return wrong data), B7 (IndexedStore rollback only on OOM), B9 (SetSize), then the remaining lows; O1/O2 remain the main scalability items.

1. Bugs (open, partly fixed, new; R10- items from Round 10)

ID	Severity	Status	Location	Problem	Suggested fix
N3	Medium	New	BufferedFileStream.pas ~389-405 (TReadOnlyCachedFileStream.Read)	On a failed cache refill FCacheStart is moved but FCacheEnd keeps its old value. A later Read can then treat the stale range as valid and return bytes from the wrong file offset, with no error.	Set FCacheEnd := FCacheStart before refilling (invalidate the window).
B7	Medium	Open	IndexedStore.pas 613, 797, 1182 and _AddIndex	Rollback runs only for EOutOfMemory. Any other exception from CompareItems (e.g. CompareFields type mismatch in lookaside nodes) leaves an item partly indexed with FCount not incremented, or a half-built index registered; on the async path the index is then freed under live links.	Catch Exception (not just EOutOfMemory) for rollback; re-raise.
B9	Medium	Open	BufferedFileStream.pas 252 (SetSize)	Seek only updates FPosition, so SetEndOfFile acts at whatever the OS pointer is; the write cache isn't flushed and FFileSize isn't updated. Latent (not called by MemDB2).	FlushCache; FileSeek(NewSize); SetEndOfFile; update FFileSize and invalidate the read cache.
B14 / R10-Q25	Low	Partly fixed	MemDB2Streamable.pas 484-500, 827-857	FIXED: FromStream nils FDataRec.Data after the FreeMem, so the double free is gone (not resetting size is fine: the object is never read again). REMAINING: the list's FromStream doesn't release NewItem when its FromStream raises, so a failed load leaks it; RecFromStream tests Rec.Data, which overlaps the integer fields (harmless on the freshly zeroed records it is called with today).	Release NewItem in an except block in TMemStreamableList.FromStream.
B10	Low/Med	Open	IndexedStore.pas 1021 (PerformAsyncActions)	If ExecParallel raises an exception outside the local filter, the cleanup tail never runs and ForceClearAsyncFlags then exposes dangling FIndexLink pointers. Async path not used by MemDB2.	try/finally around ExecParallel that always runs the tail.
B11	Low/Med	Open	IndexedStore.pas 1657 (DeleteChildren)	With an async-pending index, RemoveItem returns rvAsyncInProgress without removing, so the loop never ends and Obj.Free runs repeatedly on the same object.	Check RemoveItem result; break/raise.
B6	Low	Partly fixed	MemDB2Streamable.pas; BufferedFileStream.pas Read	FIXED: all stream I/O now uses ReadBuffer/WriteBuffer (or checked Read/Write), so short reads/writes raise. REMAINING: string/list/field-name lengths are not validated (corrupt length gives EOutOfMemory, not a clean error); cached-stream read errors set FLastError but are never surfaced, so you get a generic EReadError instead of the OS reason.	Validate lengths against Stream.Size - Position; call HandleError when FileRead returns -1.
B13	Low	Partly fixed	MemDB2Streamable.pas ToStream/RecFromStream (blob)	FIXED: blobs of 2 GiB or more now fail loudly (EWriteError/EReadError, or ERangeError in debug) instead of silently truncating. REMAINING: no up-front limit in WriteField, so the error appears at commit with a generic message; expect signed/unsigned compare warnings.	Reject size > MaxInt in TMemAPITableData.WriteField with a clear message.
N4	Low	New	BufferedFileStream.pas ~377-383 (Read, cache disabled)	A negative Count is passed straight to FileRead(Count: Longword), becoming a huge value: ReadFile can overrun the caller's buffer.	if Count <= 0 then Exit(0).
N5	Low	New	BufferedFileStream.pas 287-295 (base Read/Write)	Misuse (WriteBuffer on a read-only stream etc.) raises EAssertionFailed rather than an EStreamError descendant, so stream-error handlers miss it and it looks like a code bug.	Raise EStreamError/EReadError/EWriteError.
B15	Low	Open	IndexedStore.pas 1895 (IndexInfoByOrdinal)	Checks Assigned(Tag) on the var output before it is populated; FAllowNullTags has no setter.	Drop the pre-check or check the result.
B20b	Info	Open	MemDB2Journal.pas GetInitialFileList	Split from B20 (now resolved). StrToInt64 accepts names such as '-1.dbincr' or '$10.dbincr', which would be read as a huge or hex sequence number if someone put such a file in the journal directory. Cosmetic.	Optional: reject names that aren't plain decimal digits.
B22	Low	Partly fixed	Reffed.pas 293, 328	FIXED: GetExtraInfoText no longer appends to an uninitialised Result. REMAINING: CloneByRef leaks the new list and its refs if AddRef raises.	try/except release in CloneByRef.
R10-Q17	Info	Partly fixed	MemDB2Journal.pas 404	FIXED: the 'call close to reset' comment is gone. REMAINING: 'OAction freed in outer function' (variable is now Action).	Update the comment.

Resolved since the first review

ID	Severity	Was	Now
B20	Low	GetInitialFileList: early exit skipped FindClose and leaked the DBInfo objects in RList; a failed RList.Add leaked its DBInfo.	FindClose in a finally; failed Add frees its DBInfo; outer finally calls FreeRList, now nil-safe, so all items are freed on exit or exception. (StrToInt64 point split out as B20b.)
R10-N2	Medium	AmalgamateTransactions: if TList.Create failed, the first transaction was never released, its flush event never set and HighPriorityDone never called, so an amFlushBuffers committer blocked forever.	List created inside the try; if it was never created, the handler drops the first transaction via the new ErrorDropTransaction helper.
R10-Q16	Low	StopDBLocked swallowed a DBTmp.Free failure with no message, and asserted inside a finally under FSessionLock.	The exception message is captured into FLastError and the Assert is removed.
B18	Low	InitDB compared the un-normalised path, so a second call without a trailing slash raised S_REINIT_DIFF_LOCATION.	The path is normalised (CompareLoc + AppendTrailingDirSlash) before comparing.
B1	High	Failed FlushFileBuffers/fsync was Assert-only, so ignored in release builds.	Now raises EMemDBException (S_FLUSH_FAILED) in both incremental writes and checkpoints.
B17	Low	TNullStream.Write returned 0, so WriteBuffer/CopyFrom raised.	Returns Count.
D1	High	Introduced in your interim diff: temp files still consumed a sequence number, so the restart after a crash recovery would fail to load.	Inc(FinalSeq) now only counts final files.
B2	High	InitDB retry after a failed initial load hung forever (journal thread gone, FJournalThread still assigned).	Failed load finishes the journal via DoJournalFinished(false); Initialise waits for and frees the old thread outside the queue lock, then starts a new one.
B5	Medium	Shutdown use-after-free: the self-freeing journal thread kept touching the journal after OnJournalFinished.	Thread no longer frees itself; destructor waits for and frees it; loop reads state only under the queue lock.
B19	Low	Destroying a never-initialised TMemDB asserted in debug builds.	jscCreated accepted; FreeOnTerminate assert made nil-safe.
B8	Medium	RWWLock batch end used '=' (AdmittedContended = FRatios[Reason]); lowering a ratio mid-batch meant the batch never ended.	Now '>='.
B12	Low	RWWLock: the last thread signalled FDestroyed inside FCrit, so the destructor could free FCrit before that thread released it.	TryAcquire and Release both set a local flag under the lock and signal FDestroyed after FCrit.Release.
D2	Low	PerformMultiTransactionWrite/PerformCheckpoint freed an unassigned OutputStream in finally if file creation failed.	Creation moved outside the inner try/finally.

1c. Won't fix (your decision)

ID	Severity	Location	Issue	Note
N1	Medium	MemDB2Journal.pas 790-811 (GetListAndInitialSeqs)	If the directory holds only temp files (e.g. power loss during the very first checkpoint leaves 1.dbinit_partial), RList.Count is not 0, no final .dbinit is found and load fails with 'Couldn't find sequentially numbered db files'. Startup is blocked until the temp is deleted by hand.	-
N2	Medium	MemDB2Journal.pas 1038, 1124 (RenameFile after flush)	The rename is not made durable (MoveFile without MOVEFILE_WRITE_THROUGH; no directory fsync on POSIX). After power loss a flushed amFlushBuffers file can reappear as *_partial and is then deleted by startup cleanup, losing a commit reported durable. A failed rename also still releases the transactions as if written.	-
B3	Medium	MemDB2Journal.pas PerformCheckpoint / GetListAndInitialSeqs	FIXED: the checkpoint file is now flushed before rename. REMAINING: the loader always picks the newest .dbinit with no fallback, and the rename durability issue (N2) applies here too.	-
B4	Medium	MemDB2Journal.pas (CleanupUnusedFiles only from PerformInitialLoad)	Superseded journal files are only deleted at start-up; runtime checkpoints add a full DB copy each time and never reclaim older files, so disk use grows without bound during uptime.	-
B16	Low	Trackables.pas 106	Under USE_TRACKABLES, TTrackedMemoryStream derives from TTrackProxy instead of TMemoryStream. Currently unused.	-

ID	Severity	Location	Issue	Note
R10-N5	Medium	MemDB2Journal.pas PerformMultiTransactionWrite / PerformInitialLoad	Lazy-write files are renamed from *_partial without a flush. A crash after the rename can leave a short final file, and replay (now ReadBuffer) fails on it, so the DB won't open. Related to N2 (rename not write-through).	-
R10-N15	Low	MemDB2Buffered.pas ~2299-2311 (TTidLocal.ToScratch)	Checkpoint holds FMasterRowLock (and CommitLock) while streaming each table, blocking inserts. Documented as a deliberate trade-off; see S14.	-
R10-P3	Low	Parallelizer.pas ExecParallel	When a job raises, callers only see the exception, so successful jobs' Rets are discarded (no leak since the earlier N10 fix, only lost work).	By design.
R10-L2	Low	MemDB2.pas 923, 1241	{$IF COMPLAIN_LEAKED_TXIONS} / {$IF COMPLAIN_LEAKED_SESSIONS} reference undeclared symbols, so only the else branches are live.	Kept for debugging use.
R10-Q7	Info	MemDB2.pas 253, 678, 683	S_UNEXPECTED_CLEANUP_OPS_FAILED ('rollback or cleanup failed') is also used for commit, journal-enqueue and checkpoint failures.	-
R10-L4	Info	MemDB2Test.dproj 104, 124	Neither configuration defines USE_STRIPED_LOCK or USE_WINDOWS_LOCKS, so the striped-lock and WaitForMultipleObjects paths are never compiled or tested. Relates to O8.	-

1d. Withdrawn

ID	Severity	Original claim	Why withdrawn
B21	Low	ConsolidateAndFreeCaches: pin cache not pre-sized, so OOM mid-copy could double-release.	Incorrect. SetCapacity sizes the pin cache too, and pins enter and leave a cache only with their node, so neither TakeNodesFrom ever allocates.

2. Optimisations

ID	Impact	Status	Location	Issue	Suggestion
O1 / R10-P2	**High**	**Open**	Parallelizer.pas; MemDB2Buffered PreCommit/Commit/Rollback/index builds	ExecParallel creates one OS thread per task per call, and MemDB2 queues one task per entity per phase. Hundreds of tables means hundreds of thread creations per commit phase; a CreateThread failure in ccpData forces the DB into its error state.	Persistent pool of MaxParallel workers; run inline for a single task. (Also Round 10 item P2, rated Low there.)
O2	**High**	**Open**	MemDB2Journal.pas PerformMultiTransactionWrite	Every write batch (every amFlushBuffers commit) is still a new file, now with an extra rename each. That means millions of files and slow start-up.	Rolling segment file with length + CRC32 framing per record.
O3	**Medium**	**Open**	MemDB2.pas Checkpoint + Journal PerformCheckpoint	Full DB serialised to a temp file, then copied to the .dbinit, doubling I/O and disk use.	Serialise straight to the *_partial file and rename (the temp+rename scheme now makes this easy).
O4	**Medium**	**Open**	MemDB2BufBase.pas 1070 (RequestChangeInternal)	DeepClone under the per-object TTinyLock spinlock.	Clone outside the lock; re-lock and verify Cur unchanged.
O5	**Medium**	**Open**	MemDB2Api.pas 1695 (API_DataFindEdgeByIndex)	O(k) walk across duplicates with two format checks per step.	Lower/upper-bound search in CoWTree, giving O(log n).
O6	**Medium**	**Open**	MemDB2Journal.pas 470	Waits up to 5 s between actions even when the queue is non-empty (no high-priority pending).	Only wait when the queue is empty.
O7	**Low/Med**	**Open**	MemDB2Buffered.pas 3801	Linear skip of deleted rows while holding FMasterRowLock (acknowledged TODO).	Bounded batches / drop and re-take the lock.
O8	**Low/Med**	**Open**	StripedLock.pas	Per-row heap TStripedLock object; 65,536 critical sections at unit init.	Inline the key; fewer padded stripes; lazy init.
O9	**Low**	**Open**	BufferedFileStream.pas	Large requests double-copied through the cache; write stream's read side uses a 16 KB cache.	Bypass for large requests; pass FILE_CACHE_SIZE.
O10	**Low**	**Open**	MemDB2Buffered.pas 1109	ParallelAddHandler grows five arrays by one each call (quadratic).	Pre-size.
O11	**Low**	**Open**	MemDB2Buffered.pas 7809	EntitiesByName linear scan with pins on every lookup.	Per-transaction name map.
O12	**Low**	**Open**	MemDB2Misc.pas 768	NeverSleepOnMMThreadContention forced globally.	Make it opt-in.
O13	**Low**	**Open**	TinyLock.pas	No backoff; Sleep(0) priority-inversion risk.	Exponential backoff / SwitchToThread.
O14	**Low**	**Open**	CRC32.pas / StripedLock	Byte-wise CRC to hash random GUIDs.	XOR-fold; slice-by-8 if CRC framing adopted.

3. Not worth fixing: already covered by error handling, rollback or exceptions

ID	Area	Why it's acceptable as-is
S1	MemDB2Api UserCommitCycle / UserRollbackCycle	Fail-stop design: pre-commit failures roll back index Next under the commit lock; anything after pre-commit, or a journalling failure, sets the DB error flag, which is checked synchronously under CommitLock, so memory and journal can't silently diverge. Rollback swallows the exception but flags the DB.
S2	MemDB2Journal AmalgamateTransactions / PerformMultiTransactionWrite	Every path, including exceptions, releases transactions, signals FlushFinishedEvent and balances HighPriorityDone; errors move the journal to its error state rather than killing the thread.
S3	TWriteCachedFileStream destructor flush failure	The exception propagates out of Free into PerformMultiTransactionWrite's handler and becomes DoJournalError; the handle is still closed in finally. Only the object memory leaks.
S4	BufferedFileStream constructor failure	CreateHandle raises before FHandle is assigned, FreeMem(nil) is safe, and FlushCache guards Assigned(FCache), so destruction of a partly-built object is clean.
S5	Parallelizer thread-creation failure mid-loop	finally frees (and joins) the threads already started; unstarted slots are nil. Re-raised exceptions keep the message but lose the subclass (e.g. EDivByZero becomes EIntError), which is acceptable.
S6	CoWTree insert/delete/rebalance	Exception-safe path copying: temps are released on exception and the 'consumed' flag disambiguates ownership of the supplied node. Release on nil is a no-op.
S7	TReffed TryAddRef / TryReleaseInt	CAS loop refuses to resurrect a zero refcount, so teardown races are detected and turned into an exception rather than a use-after-free.
S8	RemoveTransaction AsyncError rollback (goto txion_remove)	Skips the rollback cycle (pins leak), but only once the DB is already in its error state and must be restarted. Acknowledged TODO.
S9	Truncated final journal file	Now largely handled: writes go to *_partial files and are renamed only when complete, and startup ignores and deletes temps. Remaining gaps are N1/N2.
S10	GetAPIObject / PutAPIObjectLocal AddRef/Release window	Runs outside the lock, but entity proxies are also held by entity lists, so the refcount can't reach zero in that window (documented).
S11	PatchFreeMem	Only on XE4 Win64; byte-compares before patching, restores page protection in finally; returns false (ignored) on any mismatch.
S12	TMemDBDatabasePersistent / TMemDBTablePersistent release-mode destructors	Brute-force 'while TryRelease' loops run only at teardown with no clients; CHECK_TEARDOWN_REFS builds do the proper cleardown transaction.
S13	OOM-only leaks (StartTransaction list add, API_CreateTable/CreateIndex, GetAPIObject, CloneByRef)	Reachable only after allocation failure, where the process state is already compromised; documented as 'very unlikely'.
S14	Checkpoint holds CommitLock for the full serialise	Deliberate simplicity/safety trade-off, documented in source; commits block but stay consistent.
S15	TDLThreadQueue FDebugLocked shared flag	Debug-only check that another thread's lock can satisfy; no effect on release behaviour.
S16	TBinTree delete-by-CopyFrom node identity swap	Handled correctly for IndexedStore via TIndexNode.CopyFrom / IndexCopiedFrom re-linking; Trackables only uses internal items.
S17	FK pre-commit renamed table/index lookups returning nil	Nil results are caught by the 'same object' / AbsIdx checks, which raise before any dereference.